Database Design

Source of truth

MSSQL is the durable source of truth for customer, authentication/session and financial state. Redis is not a replacement for MSSQL and cannot independently guarantee money correctness.

Persistence uses explicit parameterized Microsoft.Data.SqlClient. SQL transaction boundaries remain visible, especially in financial locking code.

Authentication schema

database/001_authentication_schema.sql:

- Customers;
- CustomerCredentials;
- CustomerSessions;
- RefreshTokens.

Raw passwords and raw refresh tokens are never stored.

Financial accounts schema

database/002_financial_accounts_schema.sql:

- Wallets;
- BankAccounts.

Wallet invariants:

- unique (CustomerId, Currency);
- non-negative available/blocked balances;
- customer ownership FK;
- durable lifecycle;
- DECIMAL(19,4) balance.

BankAccount invariants:

- one BankAccount per Wallet;
- internal and provider AccountId remain separate;
- composite wallet/customer/currency relationship;
- provider account ID and IBAN-like value remain consistent;
- lifecycle updates use expected status + UpdatedAt compare-and-set.

Ledger/transaction schema

database/003_ledger_transaction_schema.sql defines:

- LedgerAccounts;
- FinancialTransactions;
- IdempotencyRecords;
- LedgerJournals;
- LedgerEntries.

A **FinancialTransaction** is the durable business-operation record; it is not the same concept as a Wallet row or LedgerJournal.

Stable transaction types:

1. WalletTransfer
2. BankDeposit
3. BankWithdrawal
4. Refund
5. Reversal

Lifecycle:

1. Processing
2. Completed
3. Failed
4. Reversed

Financial decimal rule

Financial amounts must fit DECIMAL(19,4):

- maximum four decimal places;
- maximum absolute amount 999999999999999.9999.

Application boundaries validate amount/balance capacity before SQL rather than relying on database overflow errors.

Durable idempotency

Identity:

```
Scope + CustomerId + IdempotencyKey
```

Wallet-transfer scope: WALLET_TRANSFER.

The request fingerprint is SHA-256 over canonical source/destination/amount values.

Behavior:

- same key + same payload -> wait/replay the same transaction;
- same key + different payload -> conflict;
- MSSQL is final authority;
- replay returns immutable transaction fields, not current wallet balances.

The posting store uses Serializable isolation with UPDLOCK, HOLDLOCK for the idempotency key range.

Wallet-transfer accounting

Wallet Liability ledger account code:

```
WALLET-LIABILITY:{walletId:N}
```

Accounting:

```
Debit   source wallet liability       amount
Credit  destination wallet liability  amount
```

This is a pure internal liability reclassification.

Atomic posting — implemented

SqlWalletTransferPostingStore performs in one MSSQL transaction:

```
BEGIN SERIALIZABLE
-> claim/replay idempotency
-> wallet row locks (deterministic GUID order)
-> validate ownership/status/currency/balance
-> calculate domain debit/credit
-> find/create ledger accounts
-> insert Processing FinancialTransaction
-> create/post LedgerJournal + entries
-> SQL aggregate Debit/Credit validation
-> update wallet balances
-> finalize transaction
-> finalize idempotency
-> COMMIT
```

Any exception before commit rolls back the complete financial state.

Debit = Credit defense in depth

Domain LedgerJournal.Post(...) validates balance. Persistence also aggregates inserted entries in SQL. Commit is forbidden unless there are at least two entries, positive totals and exact Debit/Credit equality.

Domain rehydration

Persisted entity state is materialized through controlled Restore factories rather than reflection-based mutation of private setters.

Redis state

Redis stores transient challenge state such as registration OTP. Wallet/Ledger/FinancialTransaction truth never moves to Redis.

Current status

Public POST /api/v1/transfers is **implemented** and uses the posting store through Application fraud/session orchestration. The previous documentation statement that no public transfer endpoint existed is obsolete and has been removed.

Remaining database work

- BankDeposit/BankWithdrawal persistence workflow;
- FraudEvents/manual review;
- Outbox/Inbox;
- ReconciliationRuns/ReconciliationIssues;
- AuditEvents/operational logging storage approach;
- integration/concurrency test fixtures.